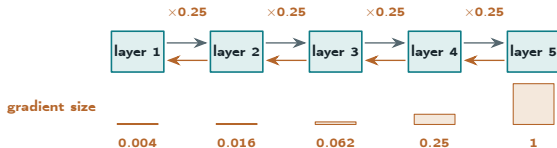


L06 The representation catalog, and learning on it

*Which 3D representation to keep depends on the questions the robot will ask of it;
which network to use depends on the representation.*

Vanishing and exploding gradients

finishing Wednesday: training, then convolutional networks



- The gradient reaching an early layer is a product of one factor for every layer after it.
- Factors below 1 shrink it: **vanishing gradients**, and early layers stop learning. With sigmoid slopes of at most 0.25, ten layers give $0.25^{10} \approx 10^{-6}$.
- Factors above 1 blow it up: **exploding gradients**, often helped along by unbounded activations, and the loss jumps to NaN.
- **Dead ReLU**: a unit whose sum is negative for every input outputs 0 with slope 0, and never updates again.

Keeping training stable

- **ReLU or a variant** in hidden layers. Leaky ReLU keeps a small slope below zero, so its units cannot die.
- **A lower learning rate:** large steps are what push ReLU units dead and gradients up.
- **Sensible starting weights**, random and scaled to the number of inputs of each layer.
- **Normalisation** of the input data, and of the outputs of intermediate layers.

Activation, learning rate and normalisation are chosen together. Change one and the best setting of the others moves.

Normalisation

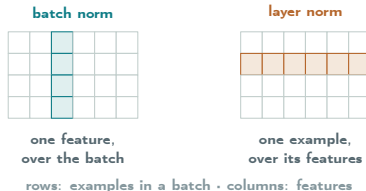
■ Batch normalisation of one feature over a batch of B examples

$$\mu = \frac{1}{B} \sum_{j=1}^B z_j, \quad \sigma^2 = \frac{1}{B} \sum_{j=1}^B (z_j - \mu)^2$$

$$\hat{z}_j = \frac{z_j - \mu}{\sqrt{\sigma^2 + \epsilon}}, \quad \text{output}_j = \gamma \hat{z}_j + \beta$$

$\epsilon > 0$ is tiny, so nothing divides by zero. γ and β are learned.

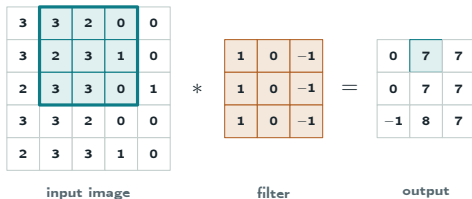
Normalisation rescales numbers to control their mean and spread. For input images: subtract the dataset mean and divide by its standard deviation, per colour channel.



Batch norm or layer norm

- **Batch norm** averages each feature over the batch. At test time there may be one image, so it uses running averages saved during training. Forgetting `model.eval()` is a classic bug.
- It degrades at small batch sizes, and 3D perception often trains with batches of 2 to 8.
- **Layer norm** averages over the features of one example, so it behaves the same at any batch size, including 1. Transformers use it.

Convolution: slide a small filter



$$(I * K)(r, c) = \sum_m \sum_n I(r + m, c + n) K(m, n)$$

- Lay the filter on a patch, multiply entry by entry, add. The shaded output is $(3 + 2 + 3) - (0 + 1 + 0) = 7$: the patch's left column minus its right. Slide one pixel and repeat.
- The same nine weights are used at every position: **weight sharing**. An edge detector works anywhere in the image.

Filters you have already met

1	2	1
2	4	2
1	2	1

blur
× 1/16

-1	0	1
-2	0	2
-1	0	1

vertical edges
Sobel

w_{11}	w_{12}	w_{13}
w_{21}	w_{22}	w_{23}
w_{31}	w_{32}	w_{33}

learned
set by backprop

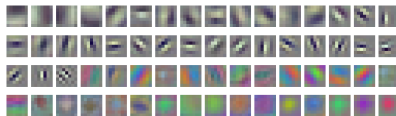
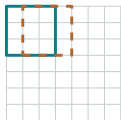


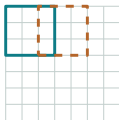
Figure: Olah et al., Distill (2020), CC BY 4.0

- L04's image pyramid blurred with a Gaussian, $G(x, y) = \frac{1}{2\pi\sigma^2} e^{-(x^2+y^2)/2\sigma^2}$, sampled on a small grid. A bigger filter or a larger σ blurs more.
- An edge filter responds where brightness changes from one side to the other.
- SIFT and ORB were built from filters like these, designed by hand. A convolutional network treats the nine numbers as weights and learns them. On the right, the first-layer filters of a trained network: edges and colour blobs.

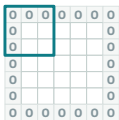
Stride and padding



stride 1
output 5 wide



stride 2
output 3 wide



padding 1
5 wide in, 5 wide out

- **Stride:** how far the filter moves each time. Larger strides give smaller outputs; above 2 is rare.
- **Padding:** a border of zeros around the input, to control the output size.

■ Output width for input W , filter K , padding P , stride S

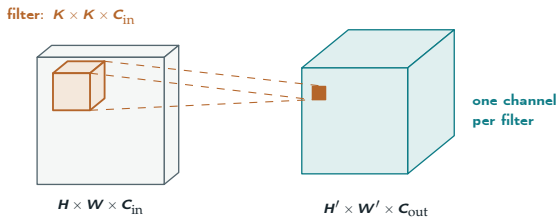
$$\text{out} = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1$$

$W = 7, K = 3, P = 0$: stride 1 gives 5, stride 2 gives 3.

$W = 224, K = 3, P = 1$: stride 1 gives 224, stride 2 gives 112.

$P = (K - 1)/2$ with $S = 1$ keeps the size unchanged.

Channels: many filters at once



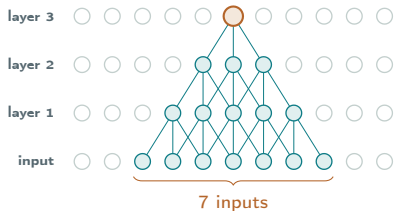
- A colour image has 3 **channels**. A filter spans all of them: $3 \times 3 \times 3$ weights and a bias.
- Use C_{out} filters and the output has C_{out} channels, one **feature map** per filter. This is the layer's **depth**.

■ **Parameters:** 3×3 filters, 3 channels in, 64 out

$$3 \cdot 3 \cdot 3 \cdot 64 + 64 = 1,792$$

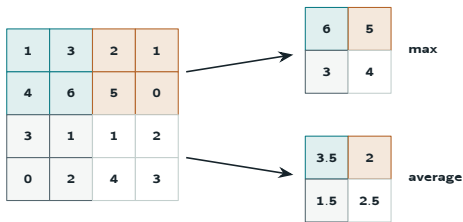
The count does not depend on the image size.

Receptive field



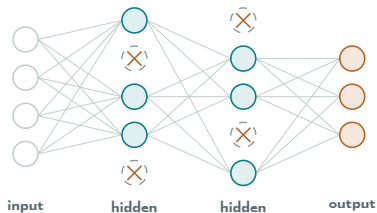
- The **receptive field** of a neuron is the patch of the input it depends on.
- Each 3×3 layer widens it by 2, so three layers see 7×7 . Stride and pooling widen it faster.
- How large it must be depends on the task. Fine geometry needs small neighbourhoods; judging motion or the layout of a room needs large ones.

Pooling



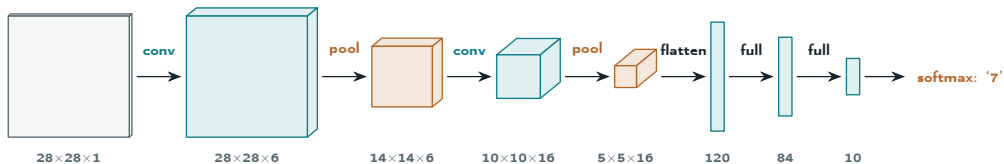
- **Max pooling** keeps the largest value in each window; **average pooling** keeps the mean.
- A 2×2 window with stride 2 halves height and width, and has no weights to learn.
- It downsamples the feature maps between layers: less computation later, and a wider receptive field.

Dropout



- During training, zero each neuron's output with probability p , often 0.5, and draw a new random set at every step.
- No neuron can rely on one particular partner being there, which reduces overfitting.
- At test time every neuron is used, with outputs scaled to match what the next layer saw in training.

A convolutional network



- Convolutions and pooling shrink height and width and add channels: more kinds of feature, each summarising a wider area.
- Then flatten, finish with fully connected layers, and softmax over the classes.
- This is LeNet (LeCun et al., 1998), which read handwritten digits. AlexNet (2012) used the same recipe, far larger, on GPUs, and won ImageNet.

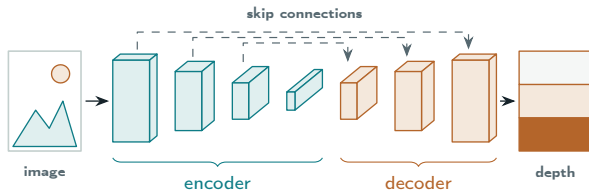
What the layers learn



Figure: Olah, Mordvintsev & Schubert, Distill (2017), CC BY 4.0

- Early filters respond to edges and colour; later layers to textures, patterns, parts, and whole objects.
- A trained CNN has learned its own image pyramid and its own feature descriptors, tuned to its task.

From a classifier to a depth network



Drop the fully connected layers. An **encoder** shrinks the image into features, a **decoder** grows them back to one output per pixel, and **skip connections** carry fine detail across (U-Net). Train it with a loss at every pixel against measured depth.

The loss decides what a network can learn. MiDaS's loss ignores scale and shift, so its depth is relative.

What you should be able to do

L05, deep learning

- Run one gradient-descent step on $\hat{y} = wx + b$ by hand, chain rule included.
- Say why a network needs a nonlinear activation, and pick one for the hidden layers and one for the output.
- Turn scores into probabilities with softmax, and score them with cross-entropy.
- Read a pair of loss curves and decide whether to keep training.
- Compute the output size and the parameter count of a convolutional layer.

To go further: 3Blue1Brown, *Neural networks*, chapters 1–4; *Dive into Deep Learning* (d2l.ai), chapters 3–5 and 7; the Stanford CS231n notes on convolutional networks.

Representing and reconstructing 3D

L06 · Unit 1 begins

What a robot can do with a 3D map depends on the representation it keeps.

Unit 0 is done. Five lectures ago a camera was a matrix; now you can write down where a robot is, what its sensors measured, and how a network is fitted to examples. Unit 1 is about what to build from those measurements.

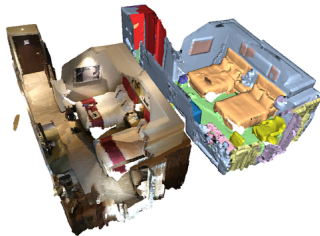


Figure: Minkowski Engine documentation, MIT License

Today

1. **What the robot asks of a map:** the questions that choose the representation.
2. **The catalog:** point clouds, voxels and octrees, signed distance, meshes, implicit functions, 3D Gaussians.
3. **Networks on point sets:** PointNet, and why a function of a set must ignore the order of its points.
4. **PointNet++:** neighbourhoods and a hierarchy, as a CNN has.
5. **Sparse convolution:** MinkowskiNet, and convolution that skips empty space.

Reading: PointNet (today's response), PointNet++ and MinkowskiNet. The catalog is a survey: TSDF fusion, the ESDF and neural fields come back on Wednesday, Gaussians in L09.

What does the robot ask of its map?

Task	Question put to the map	What the map must hold
Avoid collisions	Is this sphere free? How far is the nearest obstacle?	free space, and distance
Localise	Where does this new scan fit?	geometry to align against
Grasp	Where is the surface, and which way does it face?	surface and normals, at millimetres
Predict a view	What would the camera see from here?	appearance
Explore	What have I not seen yet?	unknown kept apart from free
Run for hours	How large has the map grown?	memory that grows with the scene, not with time

Most systems keep more than one representation, each answering a different question.

Point clouds

- A **point cloud** is a set of N points (x, y, z) , often with a colour, an intensity or a normal. It is what LiDAR and depth cameras return.
- It has no connectivity, no surface, and no record of free space: an empty region and an unobserved one look the same.
- **Normals** come from neighbours: the direction in which the k nearest points spread least, the eigenvector of their covariance with the smallest eigenvalue. A **k-d tree** finds neighbours in about $O(\log N)$ per query.

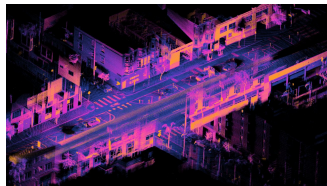
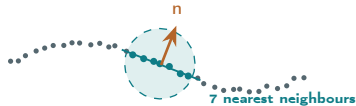


Image: Daniel L. Lu, CC BY 4.0, via Wikimedia Commons



A depth image is a point cloud on a grid

Back-project pixel (u, v) with its measured depth Z :

$$X = \frac{(u - c_x) Z}{f_x}, \quad Y = \frac{(v - c_y) Z}{f_y}$$

That is $[X \ Y \ Z]^T = Z K^{-1} [u \ v \ 1]^T$. With $f = 525$ and $(c_x, c_y) = (319.5, 239.5)$, pixel $(420, 300)$ at 1.5 m is at $(0.287, 0.173, 1.5)$ m.

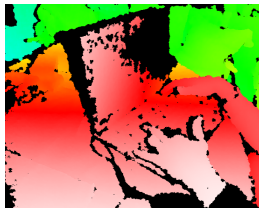
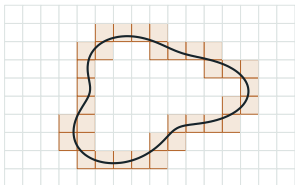


Image: Kolossos, CC BY-SA 3.0, via Wikimedia Commons

- An **organised** cloud keeps the $H \times W$ grid, so pixel neighbours are point neighbours; an **unorganised** cloud is a list.
- A 640×480 frame is 307,200 points; at 30 Hz, 9.2 million points a second.

Voxel grids



34 of 160 cells occupied; all 160 stored

- A **voxel** is a cell of a regular 3D grid. It stores occupancy, a distance, or a colour.
- Asking what is at (x, y, z) is an array index, and neighbours are adjacent cells, so 3D convolution works directly.

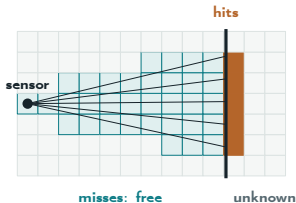
■ Memory for a $10 \times 10 \times 3$ m room, one 4-byte float per voxel

5 cm : $200 \cdot 200 \cdot 60 = 2,400,000$ voxels = 9.6 MB

1 cm : $1000 \cdot 1000 \cdot 300 = 3 \times 10^8 = 1.2$ GB

Five times finer costs $5^3 = 125$ times the memory, and most of those voxels are empty.

Occupancy is a probability



- A beam is evidence: the cells it crosses are probably free, the cell where it ends probably occupied.
- In **log-odds** $l = \log \frac{p}{1-p}$, Bayes' rule for independent measurements is a sum.

■ Update one cell with $p_{\text{hit}} = 0.7$, $p_{\text{miss}} = 0.4$, starting from $p = 0.5$

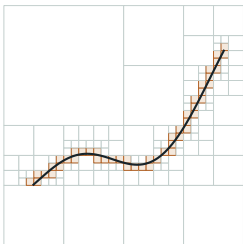
$$\text{hit: } l \leftarrow l + \log \frac{0.7}{0.3} = l + 0.85$$

$$\text{miss: } l \leftarrow l + \log \frac{0.4}{0.6} = l - 0.41$$

Three hits: $l = 2.54$, $p = 1/(1 + e^{-l}) = 0.93$.
Then one miss: $l = 2.14$, $p = 0.89$.

Clamp l to $[-2.0, 3.5]$ (p from 0.12 to 0.97), so a cell can change its mind when a door opens.

Octrees



148 leaves instead of 1024 cells

- Split a cube into eight only where something is. A large empty or unknown region stays a single node.
- **OctoMap** keeps log-odds occupancy in an octree and stores unknown separately from free, which exploration needs (L08).
- d levels of leaves of size r span $r \cdot 2^d$: 16 levels of 1 cm reach 655 m. A lookup costs $O(d)$.

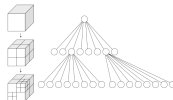
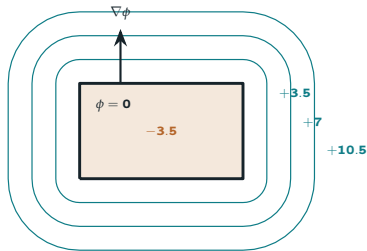


Image: WhiteTimberwolf, CC BY-SA 3.0, via Wikimedia Commons

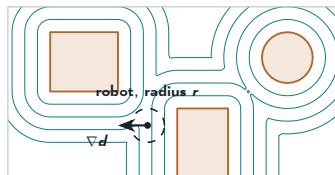
Signed distance



$$\phi(x) = \pm \min_{s \in \text{surface}} \|x - s\|$$

- Positive in free space, negative inside, zero on the surface: the surface is the level set $\phi = 0$.
- $|\nabla\phi| = 1$ almost everywhere, and $\nabla\phi$ on the surface is the normal.
- Stored on a grid, the surface falls between samples where the sign changes, so it is located to a fraction of a voxel.

TSDF and ESDF



contours of distance to the nearest obstacle

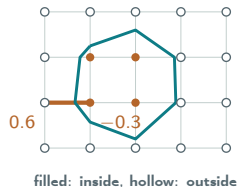
- **TSDF**: signed distance along each camera ray, clamped at $+\tau$ in front of the surface and not updated more than τ behind it, where the camera saw nothing.
- Averaging many of these in each voxel fuses depth frames (Wednesday).
- **ESDF**: the distance to the nearest obstacle, in every voxel. A robot of radius r at x is free if $d(x) > r$: one lookup.
- ∇d pushes a path away from obstacles. Voxblox builds the ESDF from the TSDF.

Meshes, and getting one from a grid

- A **mesh** is vertices and triangles: an explicit surface with connectivity, what renderers, physics engines and grasp planners take.
- It has no free space and is awkward to update, so maps are fused in a grid and meshed afterwards.
- **Marching cubes**: the signs at a cube's 8 corners give 256 cases, 15 up to symmetry, each a few triangles.



Image: Trevorgoodchild, public domain, via Wikimedia Commons



Where the surface cuts an edge with values ϕ_a and ϕ_b :

$$t = \frac{\phi_a}{\phi_a - \phi_b} = \frac{0.6}{0.6 - (-0.3)} = 0.67$$

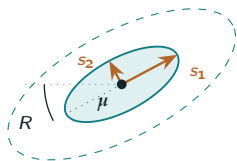
Implicit functions: a network as the map



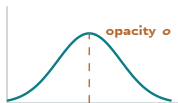
- Replace the grid with a network $f_\theta(x, y, z)$ that returns occupancy (Occupancy Networks) or signed distance (DeepSDF).
- No grid fixes the resolution; the memory is the weights.
- Every query is a forward pass, and nothing guarantees a sensible answer far from the training data.
- **NeRF** returns colour and density instead, and is the second half of Wednesday.

3D Gaussians

1 of 5: one Gaussian



solid: one standard deviation; dashed: two



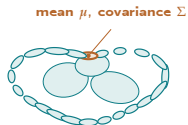
light blocked, along the long axis

$$G(x) = \exp \left(-\frac{1}{2}(x - \mu)^\top \Sigma^{-1}(x - \mu) \right)$$

- **Mean μ :** where it sits.
- **Covariance Σ :** its shape and orientation, stored as a rotation R and three scales s_1, s_2, s_3 .
- **Opacity o :** how much light it blocks at its centre; at x it blocks $o G(x)$.
- **Colour:** three numbers, or more so it can change with viewing direction.

3D Gaussians

2 of 5: a scene is millions of them



flat along the surface, large where nothing varies

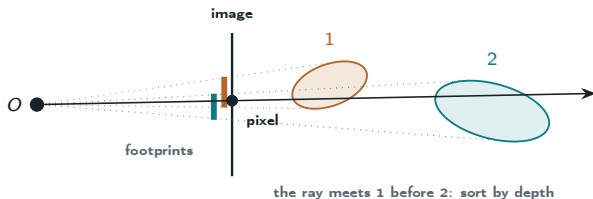
- A map fitted to photographs of one room holds a few million Gaussians.
- Thin, flat ones line up along surfaces; large round ones fill regions where nothing varies.
- Nothing connects them: no grid and no mesh.



Figure: Kerbl et al., 3D Gaussian Splatting (2023), CC BY 4.0

3D Gaussians

3 of 5: drawing an image



1. **Project** each Gaussian through the camera. A 3D ellipsoid lands on the image as a 2D ellipse, its **footprint**: L01's pinhole, applied to a blob.
2. **Sort** the Gaussians by depth.
3. **Blend** at every pixel, front to back. Each Gaussian takes its share of the light that is still left.

3D Gaussians

4 of 5: one pixel, by numbers

At this pixel, Gaussian 1 blocks $\alpha_1 = 0.6$ of the light and Gaussian 2 blocks $\alpha_2 = 0.5$. Each α_i is the opacity times the footprint's value at the pixel.

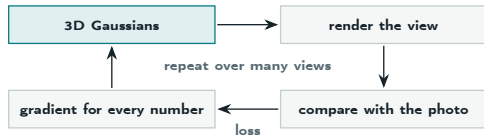
1. Gaussian 1 takes 0.6 of the pixel, and $1 - 0.6 = 0.4$ of the light gets past it.
2. Gaussian 2 takes 0.5 of what is left: $0.4 \times 0.5 = 0.2$.
3. $0.4 \times (1 - 0.5) = 0.2$ is left for whatever is behind.



$$C = \sum_i c_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j) = 0.6 c_1 + 0.2 c_2 + \dots$$

3D Gaussians

5 of 5: what the map is for

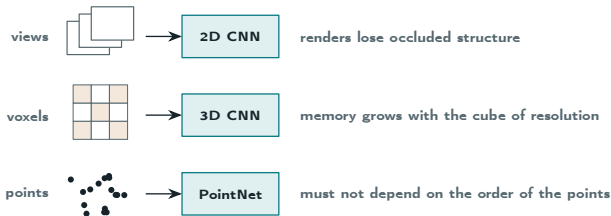


- **Fitting:** every step of the rendering is differentiable, so gradient descent (L05) adjusts every Gaussian until the renders match the photographs.
- **Good for:** new views in real time, and tracking a camera by comparing what it sees with a render.
- **Not for:** free space, surfaces or distances. A planner needs another representation beside it.
- L09 builds SLAM on Gaussians.

The catalog, side by side

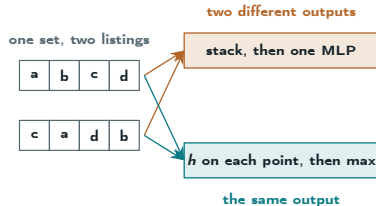
	Memory	Free space	Distance query	Online update	Typical use
Point cloud	per point	no	neighbour search	append	raw sensing, registration
Voxel grid	volume	yes	lookup	cheap	small workspaces, 3D CNNs
Octree	roughly the surface	yes, and unknown	$O(\text{depth})$ lookup	cheap	occupancy maps, exploration
TSDF, ESDF	volume, or blocks near surfaces	yes	lookup	weighted average	fusion, collision checking, planning
Mesh	surface	no	geometric search	awkward	rendering, simulation, grasping
Implicit network	the weights	learned	forward pass	retraining	shape completion
3D Gaussians	per Gaussian	no	not directly	optimisation	view synthesis, photometric SLAM

Learning directly on 3D data



- Rendered views reuse 2D CNNs but hide what is occluded, and a per-pixel answer must be carried back to 3D.
- Voxel grids let convolution work in 3D at cubic cost: 64^3 is 262,144 cells, 256^3 is 16.8 million.
- **PointNet** (Qi et al., CVPR 2017) reads the points themselves. The price is that the network must accept a set.

A function of a set



- n points can be listed in $n!$ orders; 1024 points in about 10^{2639} . The answer must be the same for all of them.
- It should also survive a few missing or extra points.

Why stacking fails, and a form that cannot fail

Two points, stacked: $W \begin{bmatrix} a \\ b \end{bmatrix} = W_1 a + W_2 b$, but $W \begin{bmatrix} b \\ a \end{bmatrix} = W_1 b + W_2 a$. Equal for every input only if $W_1 = W_2$.

Sum and max treat every point alike:
 $\max(h(a), h(b)) = \max(h(b), h(a))$.

$$f(\{x_1, \dots, x_n\}) \approx \gamma\left(\max_{i=1, \dots, n} h(x_i)\right)$$

h and γ are MLPs; the max is taken separately in each feature.

PointNet

Qi, Su, Mo and Guibas, CVPR 2017

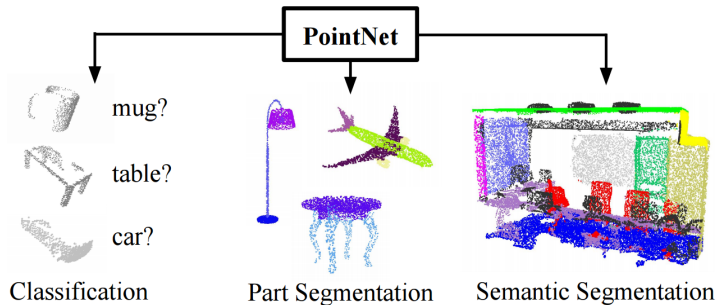
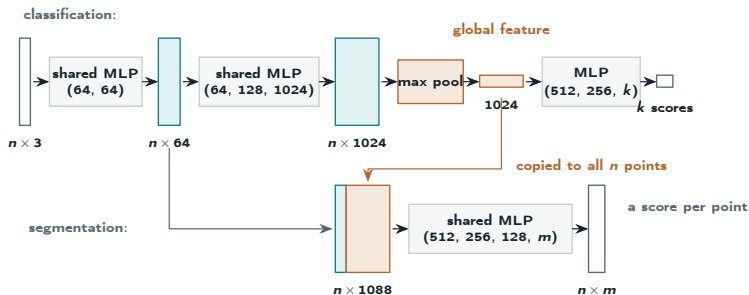


Figure: Qi et al., PointNet repository, MIT License

One architecture for classification (ModelNet40, 1024 points per shape), part segmentation (ShapeNet) and scene segmentation (Stanford indoor scans), reading raw $n \times 3$ coordinates.

PointNet, layer by layer



- **Shared MLP:** the same weights on every point, applied to each alone, so n can vary.
- **Max pool:** over the n points, in each of the 1024 features: the only step where points meet.
- **Segmentation:** a point's 64 local features plus the 1024 global ones give local and scene context.

A shared MLP is a 1×1 convolution

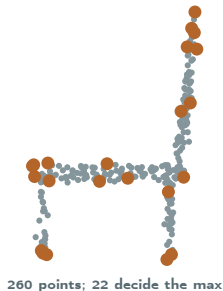
```
x = points.transpose(1, 2)           # (B, n, 3) -> (B, 3, n)
h = nn.Sequential(
    nn.Conv1d(3, 64, 1), nn.ReLU(),    # kernel size 1: each point
    nn.Conv1d(64, 64, 1), nn.ReLU(),   # is transformed on its own,
    nn.Conv1d(64, 64, 1), nn.ReLU(),   # with the same weights
    nn.Conv1d(64, 128, 1), nn.ReLU(),
    nn.Conv1d(128, 1024, 1))(x)         # (B, 1024, n)
g = h.max(dim=2).values               # (B, 1024): order of points gone
scores = classifier(g)                # MLP (512, 256, k)
```

■ Count the shared MLP's weights and biases. Does the count depend on n ?

$$(3 \cdot 64 + 64) + (64 \cdot 64 + 64) + (64 \cdot 64 + 64) + (64 \cdot 128 + 128) + (128 \cdot 1024 + 1024) = 148,992$$

No. The same weights serve 1,000 points or 100,000.

Critical points



- The max keeps, for each of the 1024 features, the one point that maximises it. At most 1024 points decide the global feature: the **critical points**.
- Remove any other point, or add one that raises no maximum, and the output does not change. With half the points missing, the paper's accuracy drops by only 2 to 4 points.
- The paper's plots of critical points look like each object's skeleton and outline: a sparse summary of the shape.

PointNet++: set abstraction

Qi, Yi, Su and Guibas, NeurIPS 2017

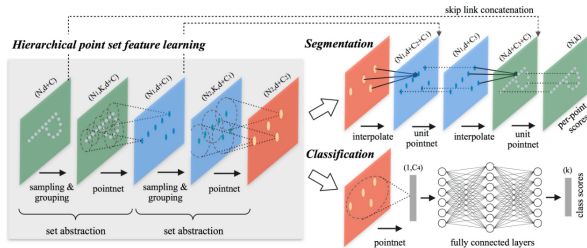
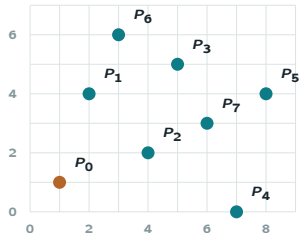


Figure: Qi et al., PointNet++ repository, MIT License

- PointNet sees each point alone, then everything at once. PointNet++ adds the gradual growth of a CNN's receptive field (L05).
- **Set abstraction:** *sample* centres by farthest point sampling, *group* the K points within radius r of each, and run a small **PointNet** on each group in local coordinates.

Farthest point sampling, by hand



Each new sample is the point farthest from every sample so far. It covers the cloud evenly; random sampling piles samples where points are dense.

■ Start from P_0 and pick three more

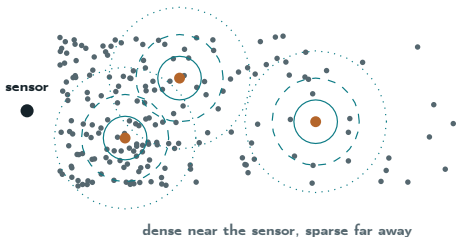
Min squared distance to $\{P_0\}$ for the rest:
10, 10, 32, 37, 58, 29, 29. Largest: P_5 (58).

Min squared distance to $\{P_0, P_5\}$ for the rest:
10, 10, 10, 17, 29, 5. Largest: P_6 (29).

Min squared distance to $\{P_0, P_5, P_6\}$ for the rest:
5, 10, 5, 17, 5. Largest: P_4 (17).

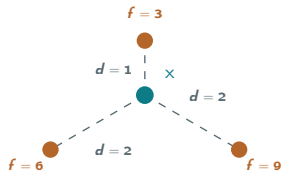
Samples in order: P_0, P_5, P_6, P_4 .

Density varies, so group at several radii



- LiDAR density falls with range (L05's rings): a fixed radius holds dozens of points near the sensor and two far away.
- **Multi-scale grouping:** run PointNets at several radii around each centre and join their features.
- **Random input dropout:** in training, delete a random fraction of the points, so the network meets many densities.

Back to every point: feature propagation



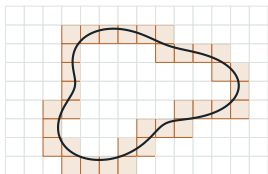
■ Interpolate a feature at x from its 3 nearest coarse points, weights $w_i = 1/d_i^2$

$$f(x) = \frac{\sum_i w_i f_i}{\sum_i w_i}$$

$$w = 1, \frac{1}{4}, \frac{1}{4} : \quad f = \frac{1 \cdot 3 + \frac{1}{4} \cdot 6 + \frac{1}{4} \cdot 9}{1.5} = 4.5$$

Segmentation needs an answer at every original point. Interpolate the coarse features back up level by level, join each with the features saved at the matching set abstraction (**skip links**), and apply a shared MLP. It is L05's encoder-decoder with skip connections, for point sets.

Most of a 3D grid is empty



34 of 126 cells occupied; all 126 stored

Surfaces are two-dimensional. A scan fills a thin shell of the grid, a few percent of the voxels or less.

■ One $3 \times 3 \times 3$ convolution, 64 channels in and out, on the 10 m room at 5 cm, with 60,000 voxels occupied

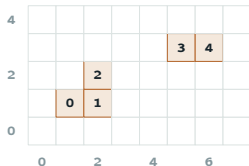
$27 \cdot 64 \cdot 64 = 110,592$ multiply-adds per output voxel

dense: $110,592 \times 2,400,000 = 2.7 \times 10^{11}$

sparse: $110,592 \times 60,000 = 6.6 \times 10^9$

40 times less work. At finer voxels the gap widens: the occupied shell grows with area, the grid with volume.

A sparse tensor



Row	Coordinates (b, x, y)	Features
0	(0, 1, 1)	$f_0 \in \mathbb{R}^C$
1	(0, 2, 1)	$f_1 \in \mathbb{R}^C$
2	(0, 2, 2)	$f_2 \in \mathbb{R}^C$
3	(0, 5, 3)	$f_3 \in \mathbb{R}^C$
4	(0, 6, 3)	$f_4 \in \mathbb{R}^C$

- Store only occupied voxels: an $N \times (D + 1)$ integer **coordinate** matrix, batch index first, and an $N \times C$ **feature** matrix.
- A hash table maps a coordinate to its row, so finding a neighbour is one lookup.
- **Quantisation:** points that fall in the same voxel are merged, so the voxel size limits the resolution.

Sparse convolution

$$x_u^{\text{out}} = \sum_{i \in \mathcal{N}(u, C^{\text{in}})} W_i x_{u+i}^{\text{in}} \quad \text{for } u \in C^{\text{out}}$$

- C^{in} and C^{out} are the occupied input and output coordinates.
- Sum only over kernel offsets i whose input site $u + i$ is occupied, and compute outputs only at the sites in C^{out} .
- W_i is one $C_{\text{out}} \times C_{\text{in}}$ matrix per offset: 27 of them for a $3 \times 3 \times 3$ kernel, as in a dense convolution.

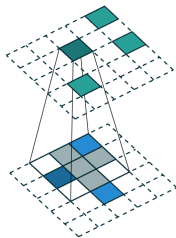
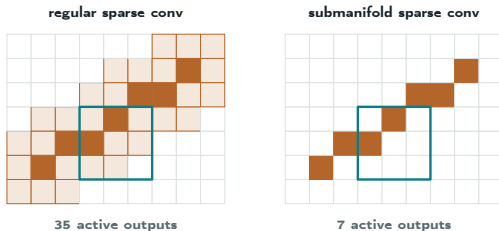


Figure: Minkowski Engine documentation, MIT License

Which sites get an output?



- **Regular** sparse convolution outputs wherever the kernel touches an occupied input, so the active set dilates, layer after layer, until the grid is no longer sparse.
- **Submanifold** sparse convolution (Graham et al., 2018) outputs only where the input was occupied: $C^{\text{out}} = C^{\text{in}}$.
- Stride 2 merges sites into a coarser grid, the role pooling plays in a 2D CNN.

MinkowskiNet

Choy, Gwak and Savarese, CVPR 2019

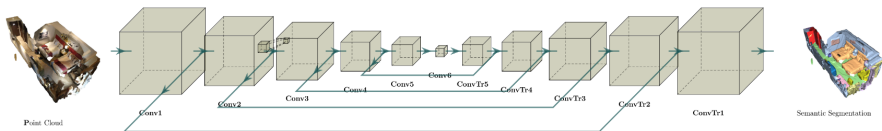


Figure: Minkowski Engine documentation, MIT License

- The **Minkowski Engine** implements generalised sparse convolution in any dimension, including 4D space-time (x, y, z, t) for sequences of scans.
- **MinkUNet** is a U-Net of sparse convolutions: strided sparse convolutions down, transposed ones up, skip connections across. It is L05's encoder-decoder, on a sparse grid.
- In 2019 it set the state of the art on ScanNet 3D segmentation, and sparse U-Nets remain standard backbones for LiDAR segmentation and 3D detection.

Points or voxels?

	PointNet++	Sparse convolution
Input	exact coordinates	quantised to voxels
Neighbours	a ball query at every layer	a hash lookup at fixed offsets
Local operator	a shared PointNet on an unordered group	one weight matrix per offset, like a 2D CNN
Scale	objects, tens of thousands of points	whole scenes, millions of voxels
Weak spot	neighbour search on large clouds	resolution set by the voxel size

A third family lets each point weigh its neighbours by **attention**. Point transformers come in L10, with the models that made attention standard in 3D.

What you should be able to do

L06

- Name the questions a robot asks of its map, and pick a representation that answers each.
- Back-project a depth pixel, update an occupancy cell in log-odds, and truncate a signed distance.
- Say why a network on points needs a symmetric function, and trace PointNet's tensor shapes.
- Run farthest point sampling and inverse-distance interpolation by hand.
- Say what a sparse convolution skips, and roughly how much work that saves.

To go further: the PointNet++ and Minkowski Engine papers' figures, and the OctoMap paper (Hornung et al., 2013) for occupancy.

Next time

L07 · Registration and map representations

Wednesday: aligning scans with ICP before fusing them, then two ways to hold a map.

Volumetric: TSDF fusion and the ESDF, as Voxblox builds them. Neural: NeRF, Instant-NGP's hash grids, and NICE-SLAM.

Response due Wednesday: Voxblox. Also assigned: NeRF, Instant-NGP and NICE-SLAM. **Both diagnostic quizzes are due Sun Sep 27.**